

From chatbot to a small, honest agent

Run live at the front. Retrieval you can inspect, two tools that refuse what they cannot do, memory you can print, and a loop that observes before it answers. Deterministic on purpose, so every failure is visible.

[Live demonstration](#)[No hand-in](#)[TF-IDF retrieval](#)[Leads into the agents labs](#)[Before you start](#)[1 · retrieval first](#)[2 · the grounded prompt](#)[3 · tools](#)[4 · memory and an explicit...](#)[5 · where it breaks](#)

Before you start

This notebook is run at the front, with the `data/` folder beside it. It deliberately uses no language model: every decision the agent makes is a line of code you can read.



By the end of this session you can

- Say what separates an agent from a chatbot: tools, state, and a loop.
- Explain retrieval-augmented generation as the answer to "it does not know my documents".
- Name where an agent can fail, before building one in the agents component.

STEP 1

retrieval first

1

Five short course documents, made searchable with TF-IDF so that every score can be inspected.

in the notebook

PYTHON

```
from pathlib import Path
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

paths = sorted(Path("data/rag_docs").glob("*.txt"))
docs, names = [p.read_text(encoding="utf-8") for p in paths], [p.name for p in paths]
vectorizer = TfidfVectorizer(stop_words="english")
D = vectorizer.fit_transform(docs)

def retrieve(query, k=2):
    scores = cosine_similarity(vectorizer.transform([query]), D)[0]
    return [{"source": names[i], "score": round(float(scores[i]), 3), "text": docs[i]} for i
in np.argsort(scores)[::-1][:k]]

for q in ["Can students use AI to write lab code?", "Do students train a transformer from
scratch?", "What is the capital of Ghana?"]:
    print(q, ">", [(h["source"], h["score"]) for h in retrieve(q)])
```

what it should print

OUTPUT

```
Can students use AI to write lab code? → [('ai_coding_policy.txt', 0.52),
('labs_and_scaffolding.txt', 0.146)]
Do students train a transformer from scratch? → [('transformer_scope.txt', 0.388),
('ai_coding_policy.txt', 0.049)]
What is the capital of Ghana? → [('transformer_scope.txt', 0.0), ('labs_and_scaffolding.txt',
0.0)]
```



Look at the third query. Retrieval always returns *something*. A score near zero is the retriever saying it found nothing relevant — and an honest system has to act on that, not paper over it.

STEP 2

the grounded prompt

2

Retrieval supplies text; the prompt tells a model to use **only** that text, to cite it, and to say when it is not enough.

in the notebook

PYTHON

```
def make_rag_prompt(question, k=2):
    context = "\n\n".join(f"[SOURCE: {h['source']}] \n {h['text']}" for h in retrieve(question,
k))
    return ("Use only the supplied context. Cite the source label for every claim. "
            "If the context is insufficient, say so instead of guessing.\n\n"
            f"QUESTION:\n{question}\n\nCONTEXT:\n{context}\n\nANSWER:")

print(make_rag_prompt("Why is accuracy alone not enough?"))
```

what it should print

OUTPUT

Use only the supplied context. Cite the source label for every claim. If the context is insufficient, say so instead of guessing.

QUESTION:

Why is accuracy alone not enough?

CONTEXT:

[SOURCE: evaluation_and_baselines.txt]

Why accuracy alone is insufficient. On imbalanced data a model can reach high accuracy by always predicting the majority class. Always compare a model against a trivial baseline, and report precision, recall and the confusion matrix. The decision threshold should be chosen from the costs of false positives and false negatives, not left at 0.5 by default.

[SOURCE: transformer_scope.txt]

Students do not train a transformer from scratch in CSC 4315. Building one would not fit in eighty minutes. Instead students compute one attention head by hand in session 25 and, in session 26, are given a small pretrained language model that runs on a laptop CPU, which they sample from and fine-tune on a small corpus.

ANSWER:

STEP 3

tools

3

Some things should never be guessed. A calculator that only evaluates arithmetic, and a lookup over the session table from the course map.

in the notebook

PYTHON

```
import ast, operator as op
import pandas as pd

OPS = {ast.Add: op.add, ast.Sub: op.sub, ast.Mult: op.mul, ast.Div: op.truediv, ast.Pow:
op.pow, ast.USub: op.neg}
def safe_calc(expr):
    def ev(n):
        if isinstance(n, ast.Constant) and isinstance(n.value, (int, float)): return n.value
        if isinstance(n, ast.BinOp) and type(n.op) in OPS: return OPS[type(n.op)](ev(n.left),
ev(n.right))
        if isinstance(n, ast.UnaryOp) and type(n.op) in OPS: return OPS[type(n.op)]
(ev(n.operand))
        raise ValueError("unsupported expression")
    return ev(ast.parse(expr, mode="eval").body)

schedule = pd.read_csv("data/course_sessions_22_28.csv")
def lookup_session(number):
    row = schedule[schedule["session"] == int(number)]
    return "No matching session in this tool." if row.empty else f"Session {number} –
{row.iloc[0].title}. Focus: {row.iloc[0].focus}"

print(safe_calc("17 * 240 / 100"))
print(lookup_session(25))
print(lookup_session(9))
```

what it should print

OUTPUT

```
40.8
Session 25 – Attention, computed by hand. Focus: queries, keys and values on a three-token
sentence
No matching session in this tool.
```

STEP 4

memory and an explicit loop

4

Plan → act → observe → remember. The planner is three `if` statements, not a model, so you can see exactly why each tool was chosen.

```

import re

class MiniCourseAgent:
    TOOLS = {"lookup_session", "calculator", "retrieve"}
    def __init__(self):
        self.memory = []
    def plan(self, message):
        m = re.search(r"session\s+(\d+)", message.lower())
        if m: return {"tool": "lookup_session", "arg": int(m.group(1))}
        if message.lower().startswith("calculate "): return {"tool": "calculator", "arg":
message[10:]}
        if any(w in message.lower() for w in ["email", "send", "book", "pay"]): return
{"tool": None, "arg": message}
        return {"tool": "retrieve", "arg": message}
    def act(self, action):
        if action["tool"] is None: return "I have no tool that can do that, so I will not
pretend to."
        if action["tool"] == "lookup_session": return lookup_session(action["arg"])
        if action["tool"] == "calculator": return safe_calc(action["arg"])
        return [(h["source"], h["score"]) for h in retrieve(action["arg"])]
    def step(self, message):
        action = self.plan(message)
        record = {"user": message, "action": action, "observation": self.act(action)}
        self.memory.append(record)
        return record

agent = MiniCourseAgent()
for q in ["What happens in session 25?", "calculate 17 * 240 / 100", "What makes an agent
more than a chatbot?", "Send an email to my professor"]:
    r = agent.step(q)
    print(f"{r['user']:<44} → {r['action']['tool']}: {r['observation']}")
print("\nmemory holds", len(agent.memory), "records")

```

```

What happens in session 25?           → lookup_session: Session 25 – Attention,
computed by hand. Focus: queries, keys and values on a three-token sentence
calculate 17 * 240 / 100              → calculator: 40.8
What makes an agent more than a chatbot? → retrieve: [('agents_vs_chatbots.txt', 0.391),
('transformer_scope.txt', 0.0)]
Send an email to my professor         → None: I have no tool that can do that, so I
will not pretend to.

memory holds 4 records

```

where it breaks

On the board, the loop **user** → **planner** → **tool or retriever** → **observation** → **memory** → **planner** → **answer**, and at each arrow one way it fails: wrong retrieval, wrong tool, bad argument, unsupported claim, stale memory, a loop that never stops.

Discuss with your neighbour

1. Ask the agent "*what is 17% of 240?*" without the word *calculate*. Which tool does it choose, and what does that show about a keyword planner?
2. When does adding an agent make a system genuinely more useful, and when does it only make a chatbot more complicated?